

Lecture 3: Linear Regression

CS4302 Intro to Machine Learning

2025

Xin Li
School of Computer Science,
Beijing Institute of Technology

Linear model for regression, More Precisely

- **Input:** Data $Z = \{(x_1, y_1), \dots, (x_n, y_n)\}$, where $x_i \in \mathbb{R}^d$ and $y_i \in \mathbb{R}$
- **Output:** A linear function $f_\beta(x) = \beta^\top x$ that minimizes the MSE:

$$L(\beta; Z) = \frac{1}{n} \sum_{i=1}^n (y_i - \beta^\top x_i)^2$$

With these choices, the linear regression problem is sometimes called “Ordinary Least Squares” (OLS).

Linear model for regression

- **Input:** Dataset $Z = \{(x_1, y_1), \dots, (x_n, y_n)\}$
- Compute

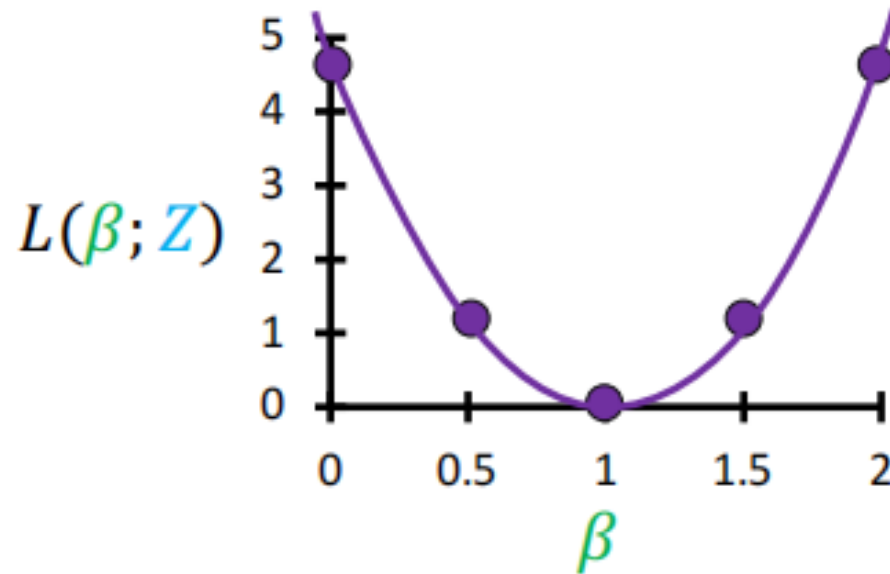
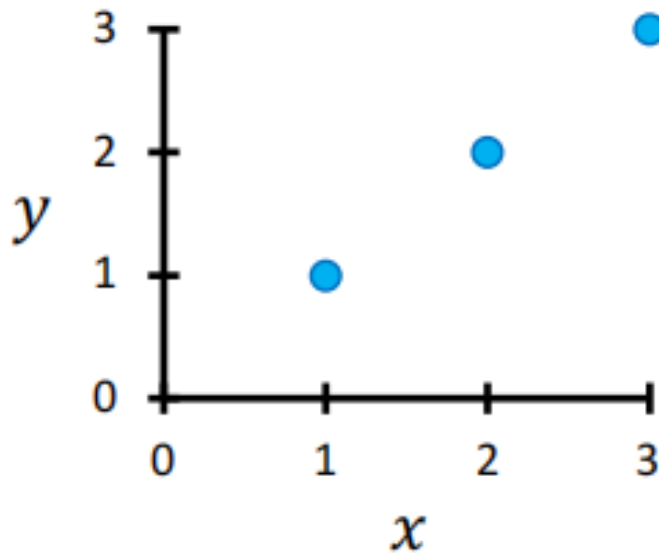
$$\begin{aligned}\hat{\beta}(Z) &= \arg \min_{\beta \in \mathbb{R}^d} L(\beta; Z) \\ &= \arg \min_{\beta \in \mathbb{R}^d} \frac{1}{n} \sum_{i=1}^n (y_i - \beta^\top x_i)^2\end{aligned}$$

- **Output:** $f_{\hat{\beta}(Z)}(x) = \hat{\beta}(Z)^\top x$

We will later discuss how to find the parameters β that minimize the MSE loss L

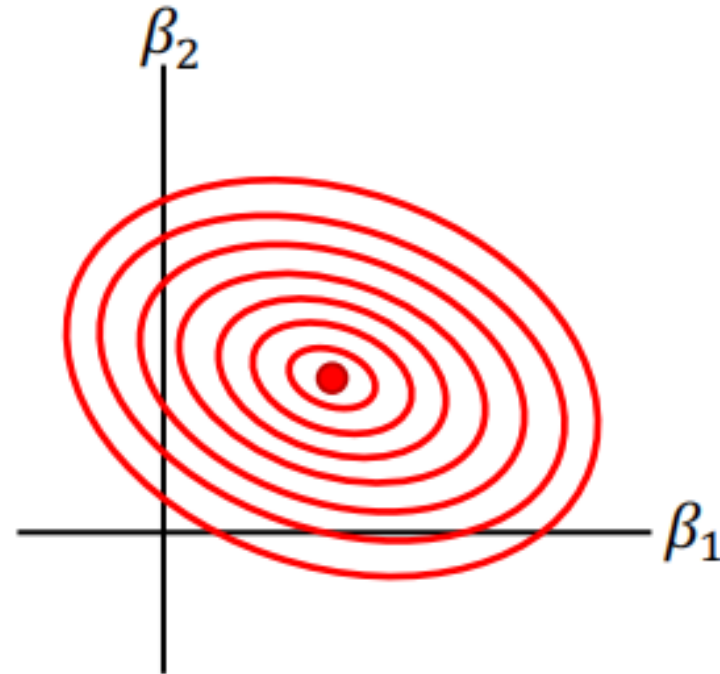
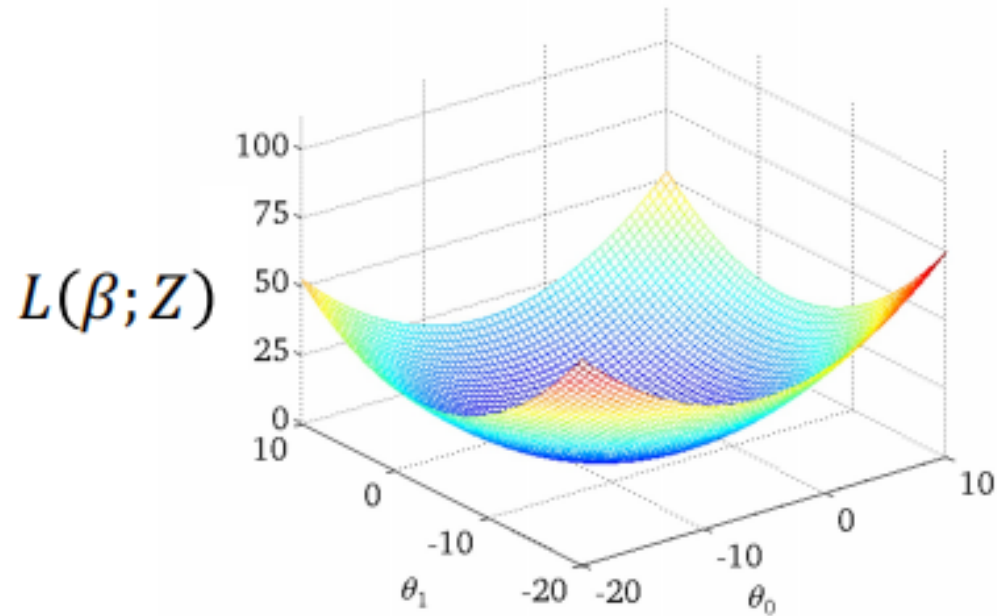
Intuition on Minimizing MSE Loss with 1-D inputs

- Consider $x \in \mathbb{R}$ and $\beta \in \mathbb{R}$, for the hypothesis class $y = \beta x$
- Then, $\text{MSE} = L(\beta; Z) = \frac{1}{n} \sum_{i=1}^n (y_i - \beta x_i)^2$



Intuition on Minimizing MSE Loss

- **Convex** (“bowl shaped”) in general



Later, we will discuss how to find the parameters β that minimize the MSE loss L

Underfitting & Overfitting



Bias & Variance

How to Fix Underfitting/Overfitting?

Three main options:

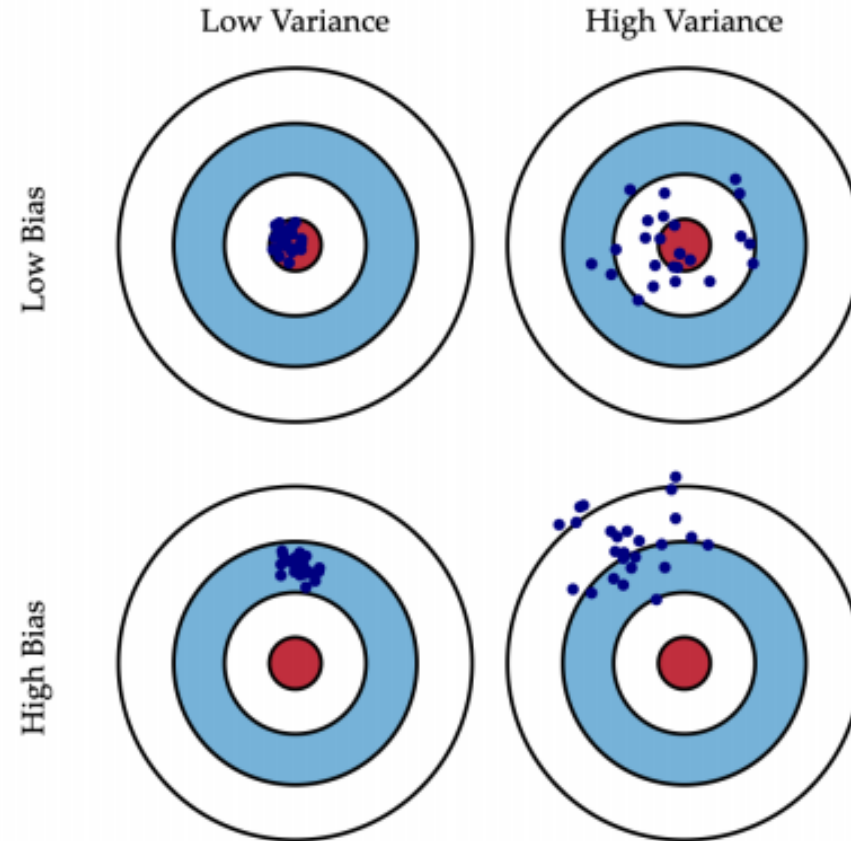
- Improve the training dataset
- Choose the right model family
- Choose the right loss function

We will explore these in some detail over the next few slides.

Definitions: “Bias” and “Variance”

Imagine you draw k training datasets from the same probability distribution over data, and each time fit your model $\{f_{\beta}\}_{1:k}$ to it.

- “**Variance**”: how much do those fitted functions $\{f_{\beta}\}_{1:k}$ differ amongst each other, on average over the data distribution?
- “**Bias**”: how much does the average of all those fitted functions $\{f_{\beta}\}_{1:k}$ deviate from the “true” function over the data distribution?



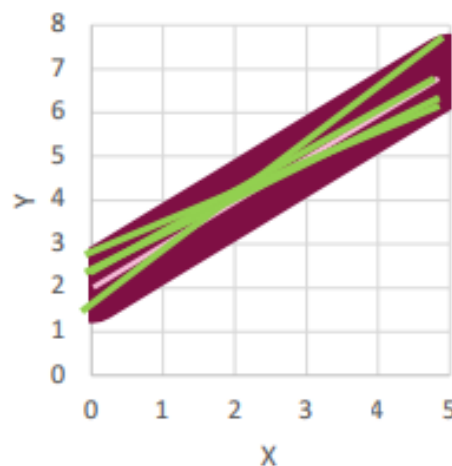
Different *Linear* Fits

Say, our hypothesis class is a line:

$$f_{\theta}(x) = \theta_0 + \theta_1 x_1$$

Fit by minimizing MSE with any optimizer. What would the resulting line look like?

Quite similar fits
“low variance”



Average fit close to the true function

“low bias”



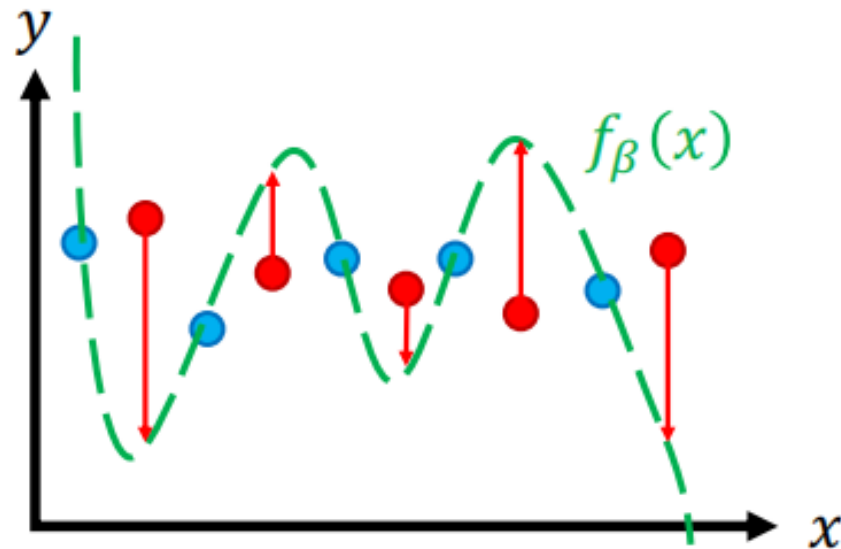
This is the type of model complexity you want to have

Theoretical result: Generalization MSE $\approx$ “Bias” + “Variance”

Bias-Variance Tradeoff

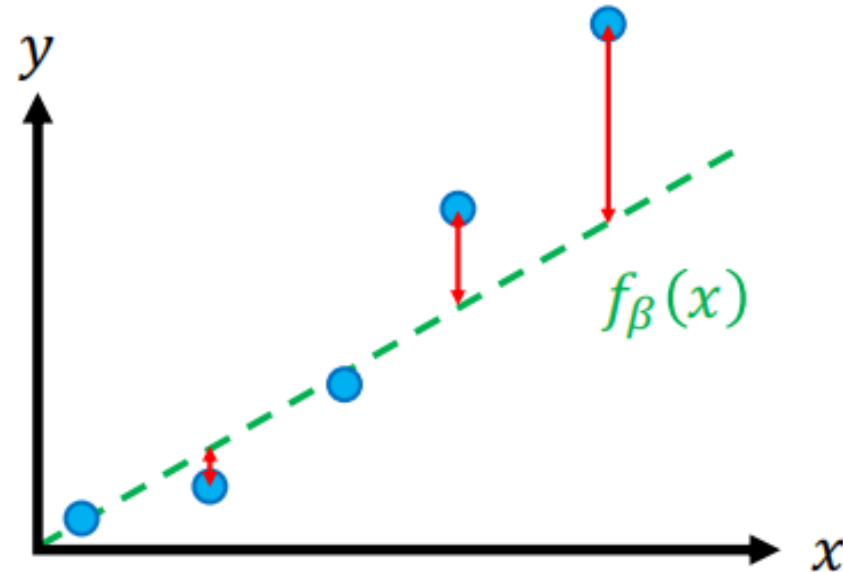
- **Overfitting (high variance)**

- High capacity model capable of fitting complex data
- Insufficient data to constrain it

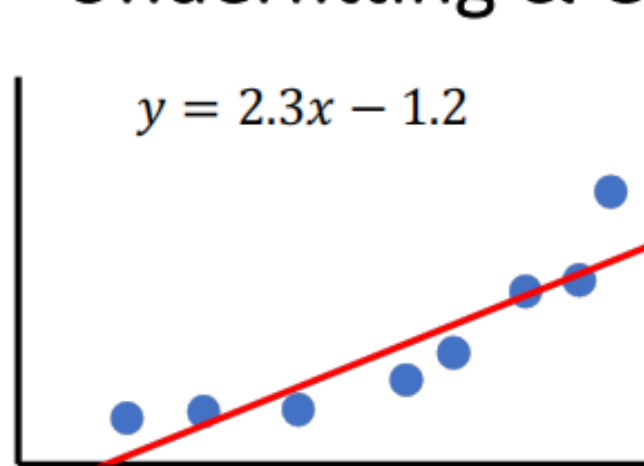


- **Underfitting (high bias)**

- Low capacity model that can only fit simple data
- Sufficient data but poor fit

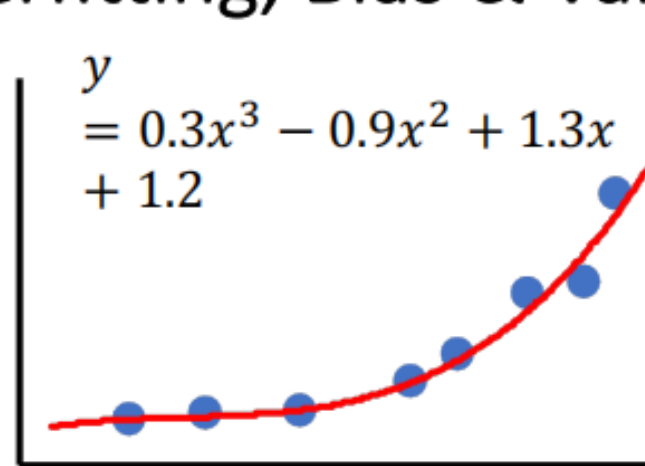


Underfitting & Overfitting, Bias & Variance



Underfitting

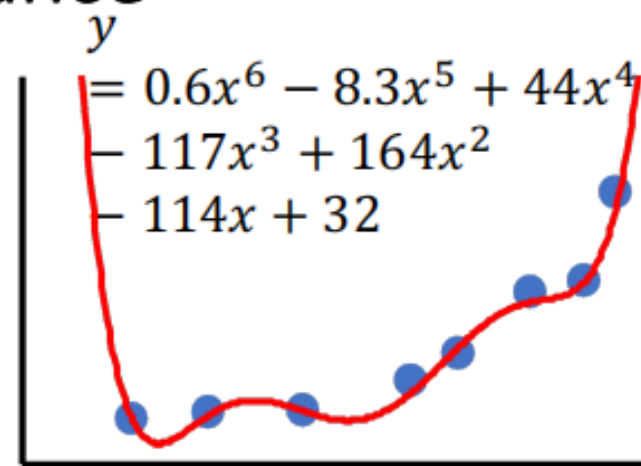
This hypothesis space has high bias



Correct fit

The data decides which hypothesis class choices are good/bad.

(Train/Test Split Protocol)



Overfitting

This hypothesis space has high variance

Bias-Variance Tradeoff For Linear Regression

- For linear regression with feature maps, increasing feature dimension d' ...
 - Tends to **increase capacity**
 - Tends to **decrease bias** but **increase variance**
- Need to construct ϕ to balance tradeoff between bias and variance
 - **Rule of thumb:** You will need $n \approx d' \log d'$ samples, if your ϕ has dimension d'
- A large fraction of data science work is data cleaning + feature engineering. We will see some common rules of thumb for feature engineering soon.

The Effect of Dataset Size

Recall, we said:

Let us draw $k \rightarrow \infty$ training sets of $n = 6$ samples each, drawn from $P(X, Y)$.

Q: What if we had drawn larger training sets? Would it impact:

- Bias?

Usually no. As $k \rightarrow \infty$, the average of fits $\{f_\beta\}_{1:k}$ to k training sets of finite size n (for any n) approaches the fit to an $n' \rightarrow \infty$ -sized set.

Often convenient to think of bias as the lowest achievable error corresponding to the “best model” within the hypothesis space.

- Variance?

Yes, as the dataset size grows, different i.i.d. datasets would induce similar fits $\Rightarrow$ lower variance.

Convenient to think of variance as average error w.r.t. the “best model”.

The Effect of Dataset Size

As dataset size grows:

- Generalization error ($\approx$ ``Bias'' + ``Variance'') is dominated by bias.
- To reduce error, we select high capacity, low bias models.

Larger datasets have room for expanded hypothesis classes.

Regularization

Regularization: Modifying the Loss function

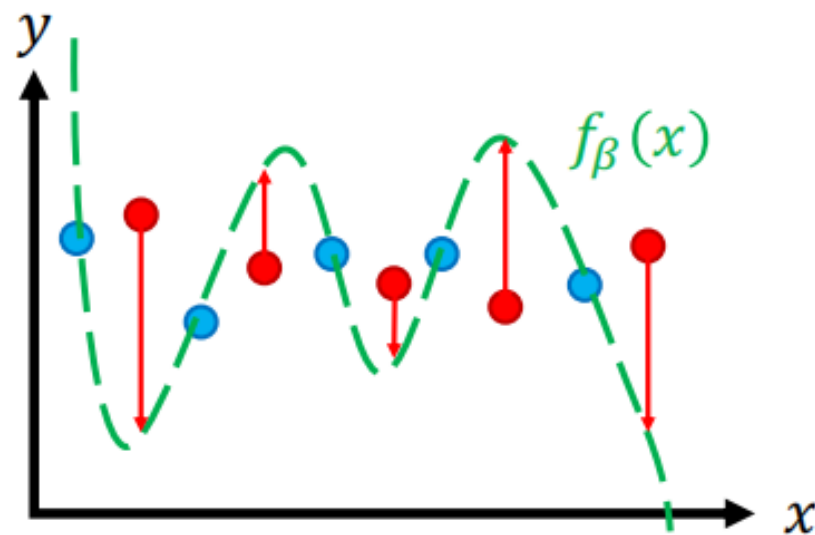
- **Intuition:** We *only* asked the ML algorithm to fit the training data as well as possible, so it produced overly complex fits → “Overfitting”

$$L(\beta; Z) = \text{Train MSE}$$

- **Solution:** we will ask the model to produce a “*simple fit*” to the training data.

$$L(\beta; Z) = \text{Train MSE} + \text{Fit complexity}$$

How to measure this?



Linear Regression with L_2 Regularization

- **Original loss** + **regularization**:

One measure of fit complexity

$$\begin{aligned} L(\beta; Z) &= \frac{1}{n} \sum_{i=1}^n (y_i - \beta^\top x_i)^2 + \lambda \cdot \|\beta\|_2^2 \\ &= \frac{1}{n} \sum_{i=1}^n (y_i - \beta^\top x_i)^2 + \lambda \sum_{j=1}^d \beta_j^2 \end{aligned}$$

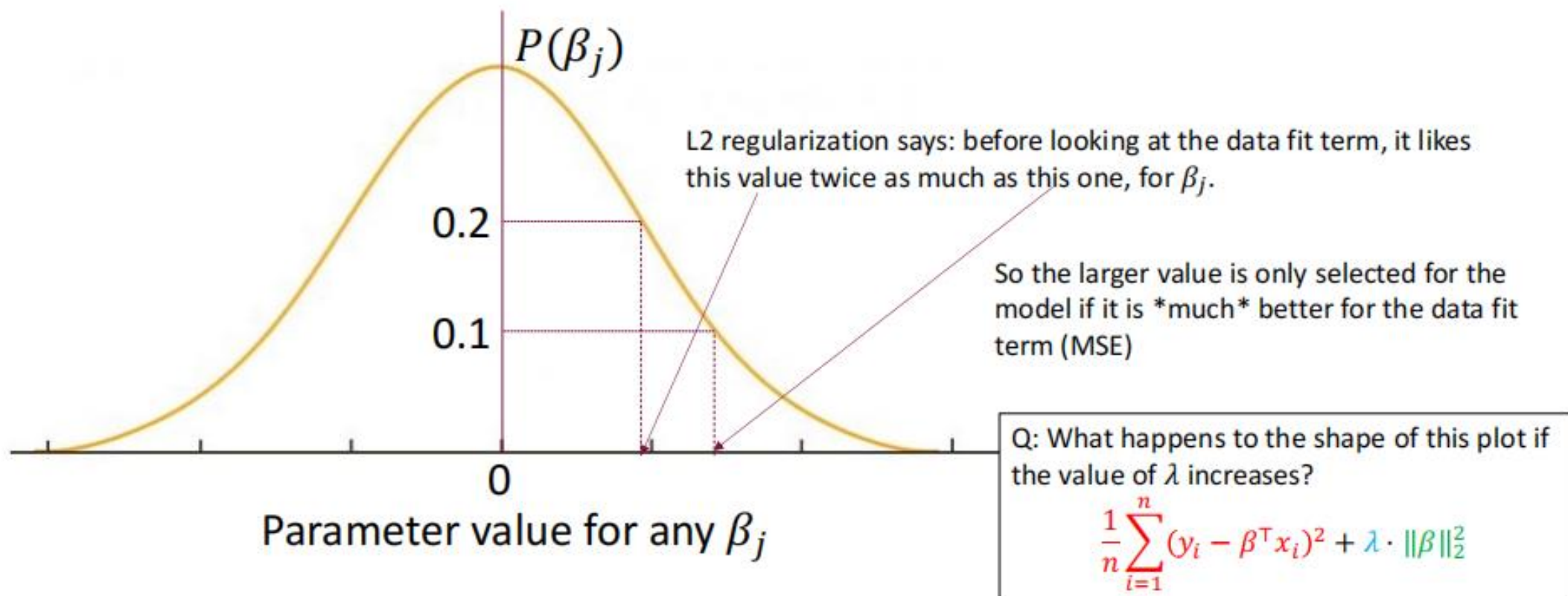

- λ is a **hyperparameter** that must be tuned (satisfies $\lambda \geq 0$)

Intuition on L_2 Regularization

- A thought experiment.
 - Consider a feature map with $d = 10$ features $[x_1, \dots, x_d]$
 - Suppose during linear regression that we forced $\beta_j = 0$ for all $j > 5$
 - This is exactly equal to choosing a smaller-capacity hypothesis class, induced by the smaller feature map with $d = 5$
- Thus, forcing β_j 's to be 0 induces a smaller-capacity hypothesis class.
- The soft version of this: encouraging β_j 's to have small magnitude also induces a smaller-capacity hypothesis class.
 - This is what L_2 regularization does: $\sum_{j=1}^d \beta_j^2 = \|\beta\|_2^2 = \|\beta - 0\|_2^2$
 - Pulls coefficients towards 0
 - As $\lambda \rightarrow \infty$, it forces $\beta = 0$

Intuition on L_2 Regularization: Gaussian Priors

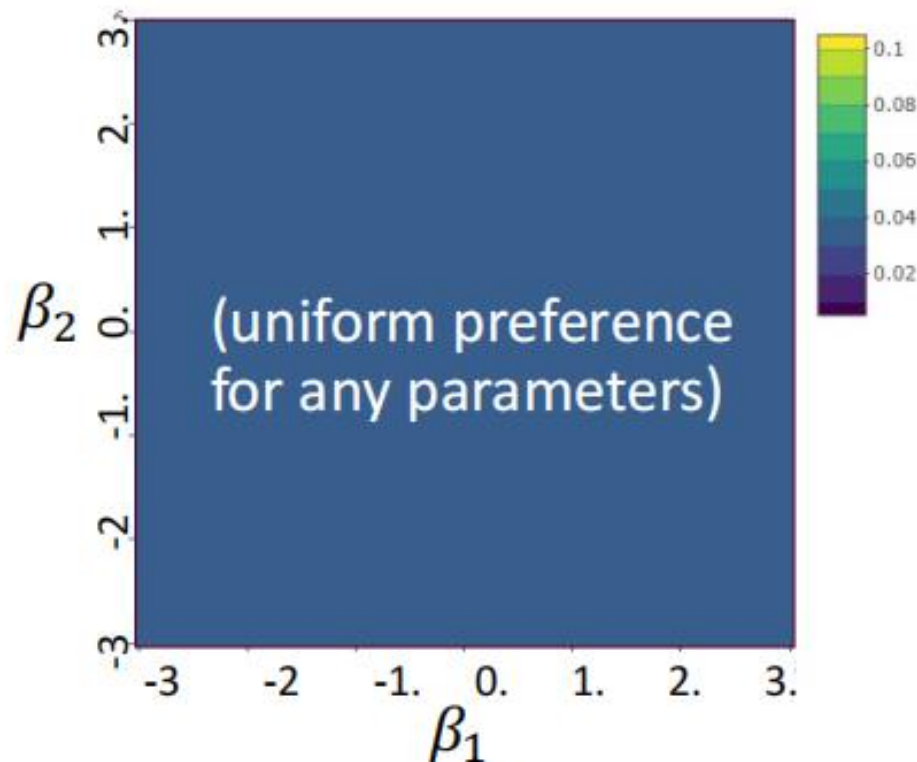
L_2 regularized linear regression amounts to preferring smaller weights according to a Gaussian pdf.



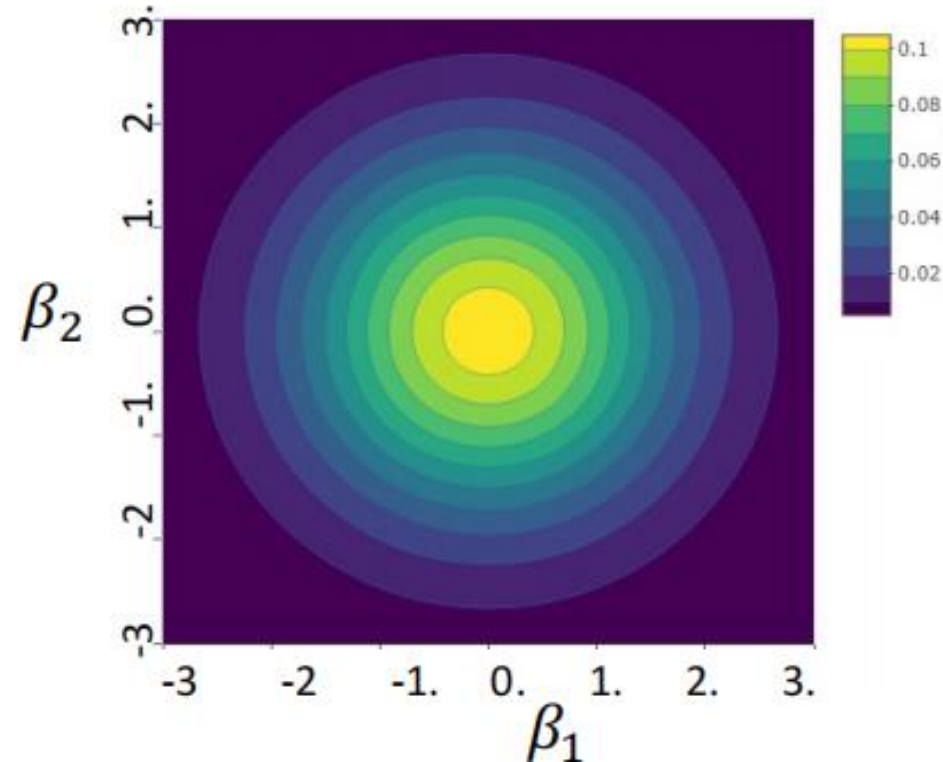
Intuition on L_2 Regularization: Gaussian Priors

- Extending the gaussian pdf over each individual weight β_j to the full parameter vector β , the hypothesis space is constrained.

Before regularization



With L2 regularization

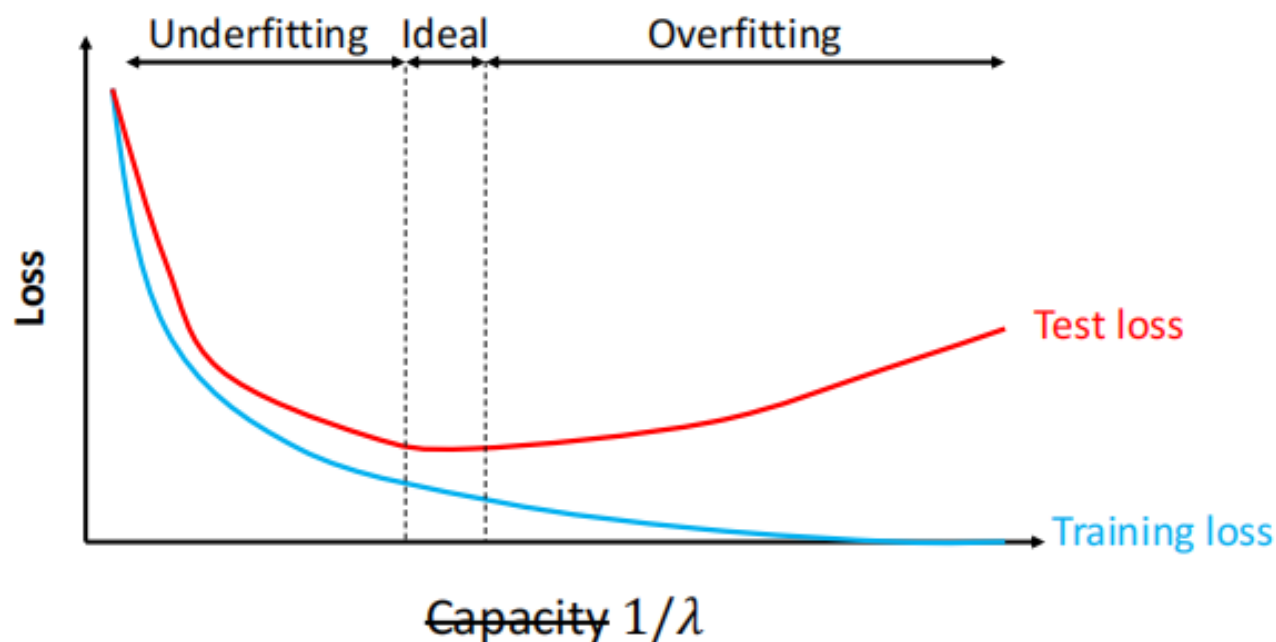


Intuition on L_2 Regularization

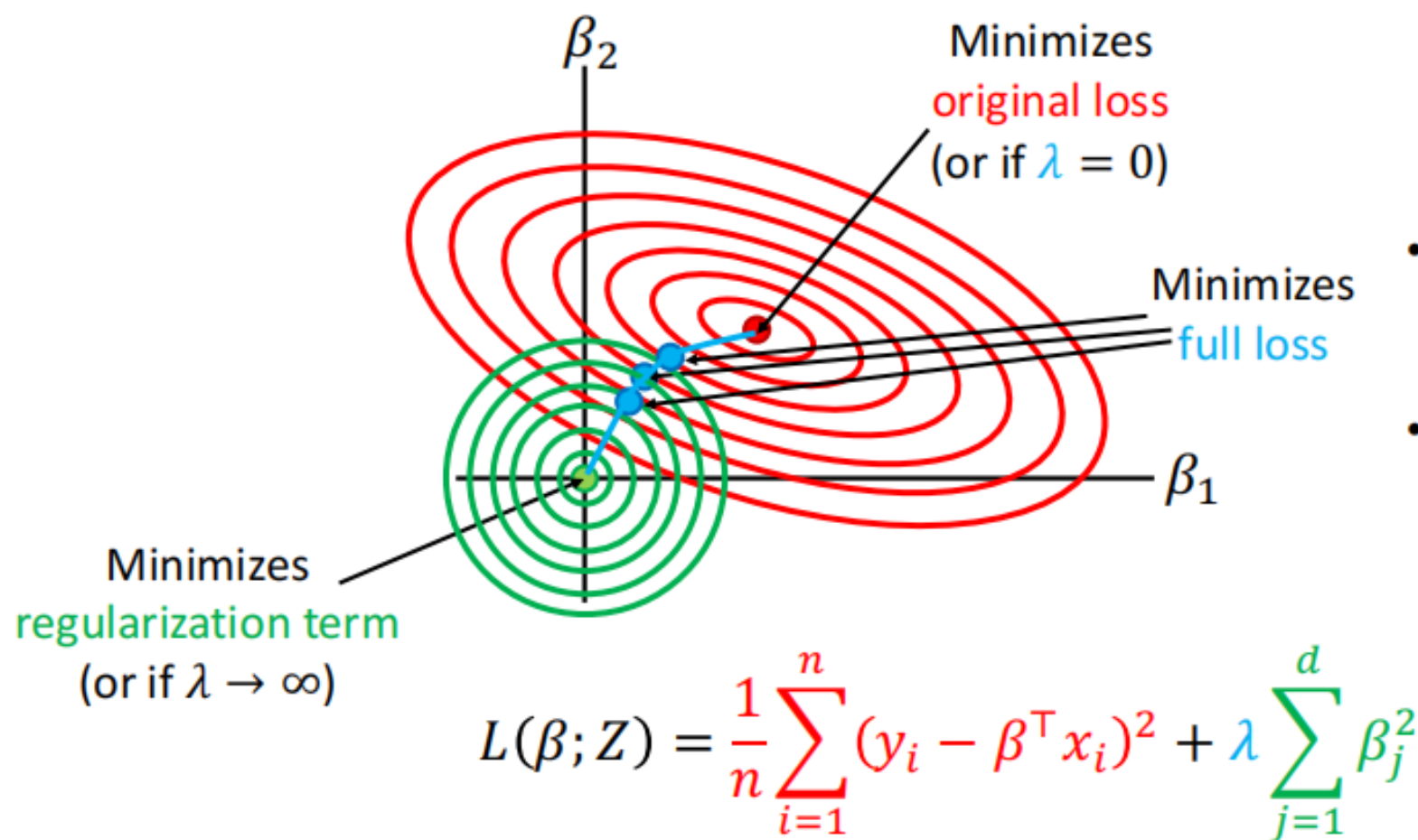
- Why does it help?

- Encourages “simple” functions
- E.g., Use λ to tune bias-variance tradeoff.

Q: How would you set λ to get higher bias / lower variance?



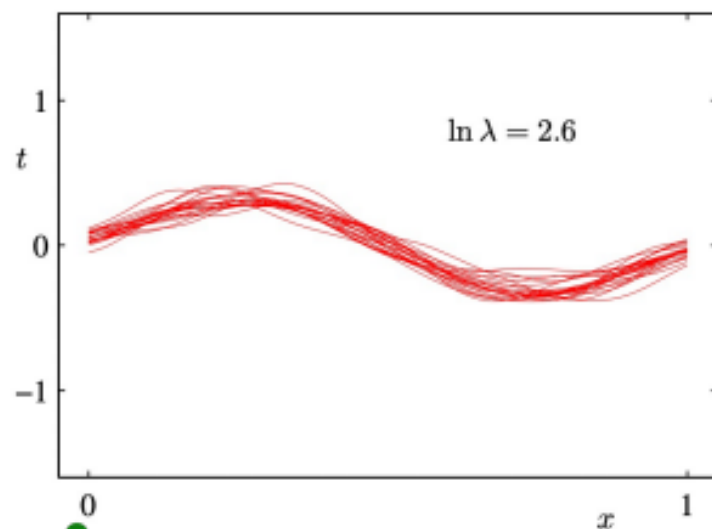
Intuition on L_2 Regularization



- At this point, the gradients are **equal** (with opposite sign)
- Tradeoff depends on choice of λ

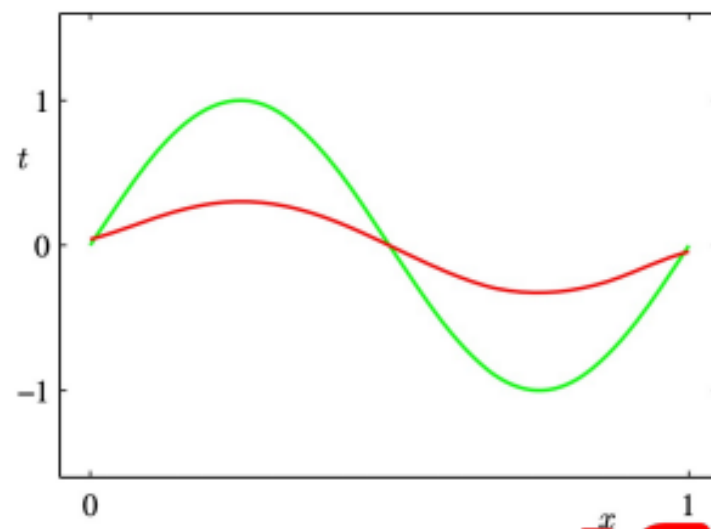
Effect of λ on Bias and Variance

Fits to various datasets sampled from an underlying sinusoid



Illustrates variance

Mean fit compared to the "true function"



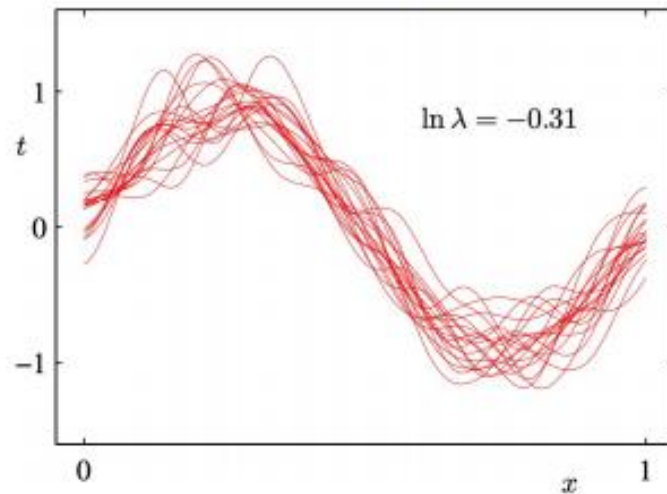
Illustrates bias

With λ too high, the algorithm finds overly simple solutions.

*dataset size n set to 25, and feature map dimensionality d' is 25 (gaussian feature map)

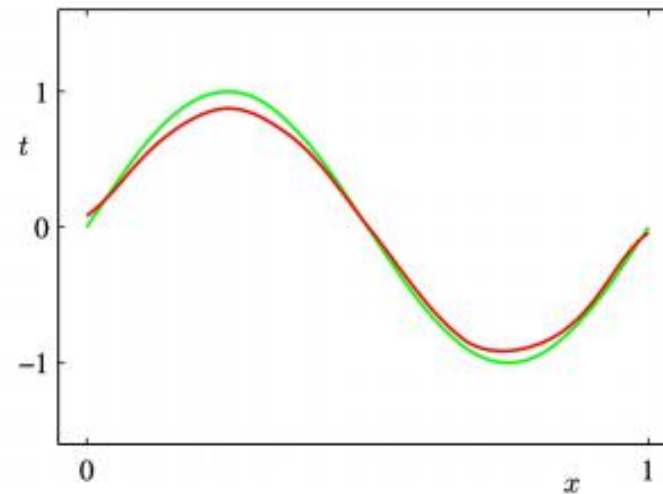
Effect of λ on Bias and Variance

Fits to various datasets sampled from an underlying sinusoid



Illustrates variance

Mean fit compared to the "true function"



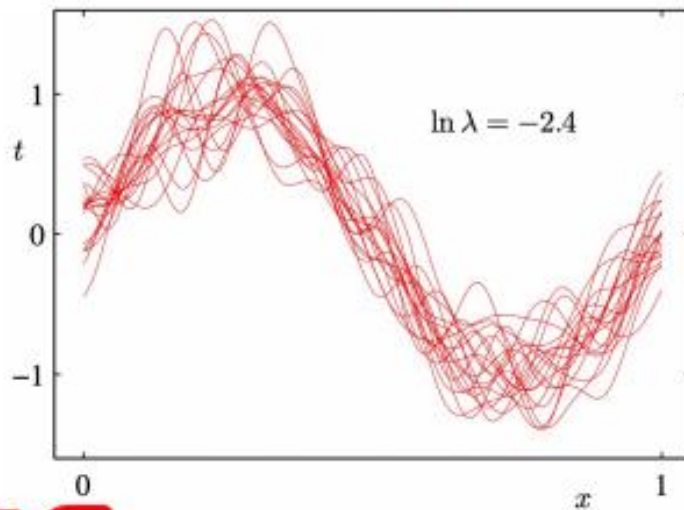
Illustrates bias



*dataset size n set to 25, and feature map dimensionality d' is 25 (gaussian feature map)

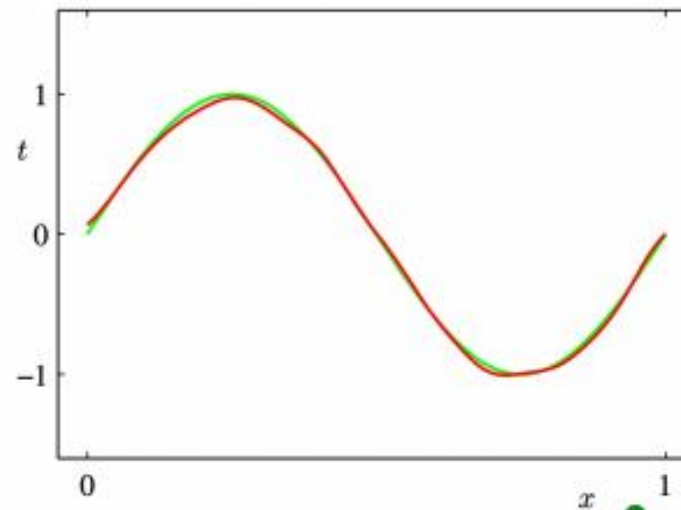
Effect of λ on Bias and Variance

Fits to various datasets sampled from an underlying sinusoid



Illustrates variance

Mean fit compared to the "true function"



Illustrates bias



With λ too low, the algorithm finds overly complex solutions.

*dataset size n set to 25, and feature map dimensionality d' is 25 (gaussian feature map)

General Regularization Strategy

- **Original loss** + **regularization**:

$$L_{\text{new}}(\beta; Z) = L(\beta; Z) + \lambda \cdot R(\beta)$$

- Offers a way to express a preference for “simpler” functions in family
- Typically, regularization is independent of data

Q: For the new parameters $\beta_{\text{new}}^* = \min_{\beta} L_{\text{new}}$, would their corresponding value of $L(\beta; Z)$ be smaller or larger than before regularization?

Vectorizing Linear Regression

$$\begin{bmatrix} f_{\beta}(x_1) \\ \vdots \\ f_{\beta}(x_n) \end{bmatrix} = \begin{bmatrix} \beta^{\top} x_1 \\ \vdots \\ \beta^{\top} x_n \end{bmatrix} = \begin{bmatrix} \sum_{j=1}^d \beta_j x_{1,j} \\ \vdots \\ \sum_{j=1}^d \beta_j x_{n,j} \end{bmatrix} = \begin{bmatrix} x_{1,1} & \cdots & x_{1,d} \\ \vdots & \ddots & \vdots \\ x_{n,1} & \cdots & x_{n,d} \end{bmatrix} \begin{bmatrix} \beta_1 \\ \vdots \\ \beta_d \end{bmatrix} = X\beta$$

$\approx$

$$\begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix} = Y$$

Summary: $Y \approx X\beta$

Note: n equations, d variables

Vectorizing Linear Regression

$$Y \approx X\beta$$

$$Y = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}$$

$$X = \begin{bmatrix} x_{1,1} & \cdots & x_{1,d} \\ \vdots & \ddots & \vdots \\ x_{n,1} & \cdots & x_{n,d} \end{bmatrix}$$

$$\beta = \begin{bmatrix} \beta_1 \\ \vdots \\ \beta_d \end{bmatrix}$$

Vectorizing Mean Squared Error

$$L(\beta; Z) = \frac{1}{n} \sum_{i=1}^n (y_i - \beta^\top x_i)^2 = \frac{1}{n} \|Y - X\beta\|_2^2$$

Diagram illustrating the vectorization of the Mean Squared Error (MSE) function:

- The vector $\begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}$ (labeled Y) is mapped to the term Y in the vectorized equation.
- The vector $\begin{bmatrix} f_\beta(x_1) \\ \vdots \\ f_\beta(x_n) \end{bmatrix}$ is mapped to the term $X\beta$ in the vectorized equation.
- The squared L2 norm $\|z\|_2^2 = \sum_{i=1}^n z_i^2$ is mapped to the term $\|Y - X\beta\|_2^2$ in the vectorized equation.

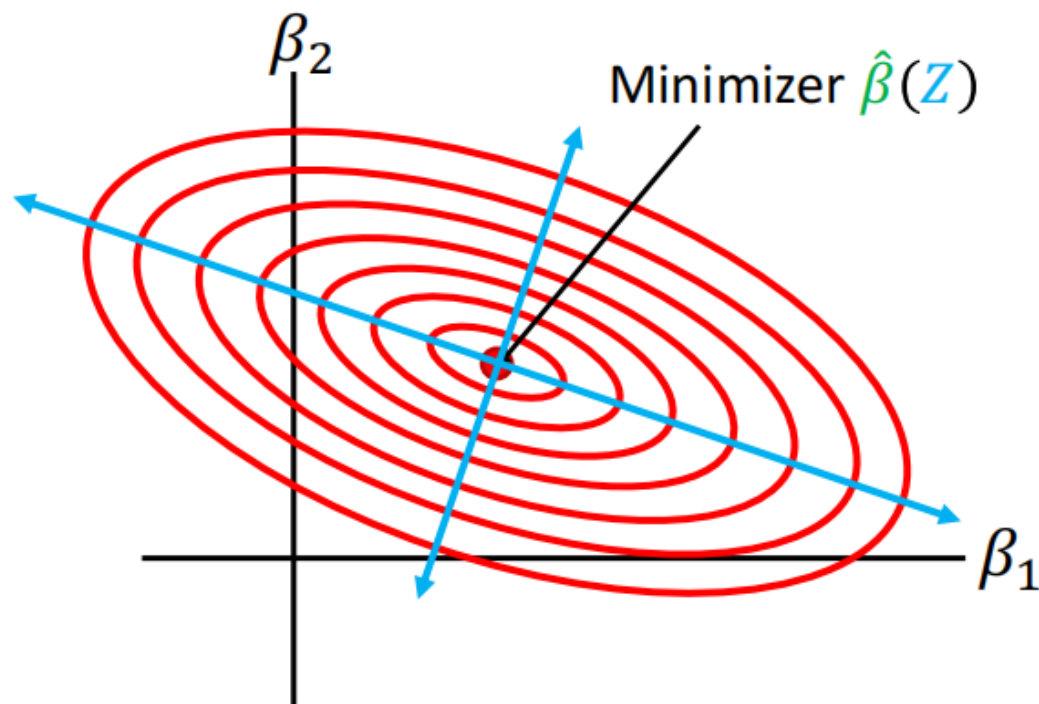
Intuition on Vectorized Linear Regression

- Rewriting the vectorized loss:

$$\begin{aligned}n \cdot L(\beta; Z) &= \|Y - X\beta\|_2^2 = \|Y\|_2^2 - 2Y^T X\beta + \|X\beta\|_2^2 \\&= \|Y\|_2^2 - 2Y^T X\beta + \beta^T (X^T X)\beta\end{aligned}$$

- **Side note:** Quadratic function of β with leading “coefficient” $X^T X$
 - In one dimension, “width” of parabola $ax^2 + bx + c$ is a^{-1}
 - In multiple dimensions, “width” along direction v_i is λ_i^{-1} , where v_i is an eigenvector of $X^T X$ with eigenvalue λ_i
 - Large width (small λ_i) along a direction v_i implies that parameter values along that direction affect the loss value less.
 - This will be interesting to us later in class (“PCA”)

Intuition on Vectorized Linear Regression



Directions/magnitudes are given by eigenvectors/eigenvalues of $X^T X$

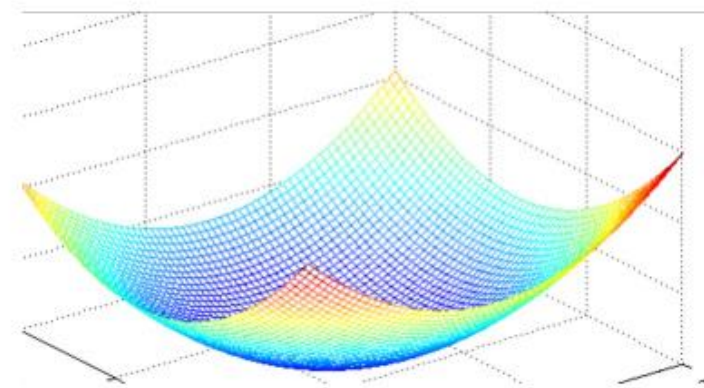
Strategy 1: Closed-Form Solution

- Recall that linear regression minimizes the loss:

$$L(\beta; Z) = \frac{1}{n} \|Y - X\beta\|_2^2$$

- Minimum solution has gradient equal to zero:

$$\nabla_{\beta} L(\hat{\beta}(Z); Z) = 0$$



Strategy 1: Closed-Form Solution

- The gradient is

$$\begin{aligned}\nabla_{\beta} L(\beta; Z) &= \nabla_{\beta} \frac{1}{n} \|Y - X\beta\|_2^2 = \nabla_{\beta} \frac{1}{n} (Y - X\beta)^{\top} (Y - X\beta) \\ &= \frac{2}{n} [\nabla_{\beta} (Y - X\beta)^{\top}] (Y - X\beta) \\ &= -\frac{2}{n} X^{\top} (Y - X\beta) \\ &= \boxed{-\frac{2}{n} X^{\top} Y + \frac{2}{n} X^{\top} X \beta}\end{aligned}$$

Strategy 1: Closed-Form Solution

- The gradient is

$$\nabla_{\beta} L(\beta; Z) = \nabla_{\beta} \frac{1}{n} \|Y - X\beta\|_2^2 = -\frac{2}{n} X^T Y + \frac{2}{n} X^T X \beta$$

- Setting $\nabla_{\beta} L(\hat{\beta}; Z) = 0$, we have $X^T X \hat{\beta} = X^T Y$

Compare this to: $Y \approx X\beta$

- Assuming $X^T X$ is invertible, we have

$$\hat{\beta}(Z) = \underbrace{(X^T X)^{-1} X^T}_{\text{pseudoinverse of } X} Y$$

This is called the
“pseudoinverse” of X

Strategy 1: Closed-Form Solution

- Setting $\nabla_{\beta} L(\hat{\beta}; Z) = 0$, we have $X^T X \hat{\beta} = X^T Y$
- Assuming $X^T X$ is invertible, we have

$$\hat{\beta}(Z) = (X^T X)^{-1} X^T Y$$

Note on Invertibility

- Closed-form solution only **unique** if $X^T X$ (size $d \times d$) is invertible
 - Otherwise, **multiple solutions exist** to $X^T X \hat{\beta} = X^T Y$
 - **Intuition:** Underconstrained system of linear equations

- **Example:**

$$\begin{bmatrix} 1 & 1 \\ 2 & 2 \end{bmatrix} \begin{bmatrix} \hat{\beta}_1 \\ \hat{\beta}_2 \end{bmatrix} = \begin{bmatrix} 2 \\ 4 \end{bmatrix}$$

- In this case, any $\hat{\beta}_2 = 2 - \hat{\beta}_1$ is a solution

When Can this Happen?

- **Case 1**

- Fewer data examples than feature dimension (i.e., $n < d$)
 - Remember: we are solving something like $Y \approx X\beta$
- **Solution:** Remove features so $d \leq n$
- **Solution:** Collect more data until $d \leq n$

- **Case 2:** Some feature is a linear combination of the others

- **Special case (duplicated feature):** For some j and j' , $x_{i,j} = x_{i,j'}$ for all i
- **Solution:** Remove linearly dependent features
- **Solution:** Use L_2 regularization (we will soon see why)

Shortcomings of Closed-Form Solution

- Computing $\hat{\beta}(Z) = (X^T X)^{-1} X^T Y$ can be challenging
- **Computing $(X^T X)^{-1}$ is $O(d^3)$**
 - $d = 10^4$ features $\rightarrow O(10^{12})$
 - Even storing $X^T X$ requires a lot of memory
- **Numerical accuracy issues due to “ill-conditioning”**
 - $X^T X$ is “barely” invertible
 - Then, $(X^T X)^{-1}$ has large variance along some dimension
 - Regularization helps (more on this later)

Iterative Optimization Algorithms

- Recall that linear regression minimizes the loss

$$L(\beta; Z) = \frac{1}{n} \sum_{i=1}^n (y_i - \beta^\top x_i)^2$$

- Iteratively optimize β
 - Initialize $\beta_1 \leftarrow \text{Init}(\dots)$
 - For some number of iterations T , update $\beta_t \leftarrow \text{Step}(\dots)$
 - Return β_T

Iterative Optimization Algorithms

- **Global search:** Try random values of β and choose the best
 - I.e., β_t independent of β_{t-1}
 - Very unstructured, can take a long time (especially in high dimension d)!
- **Local search:** Start from some initial β and make local changes
 - I.e., β_t is computed based on β_{t-1}
 - What is a “local change”, and how do we find good one?

Strategy 2: Gradient Descent

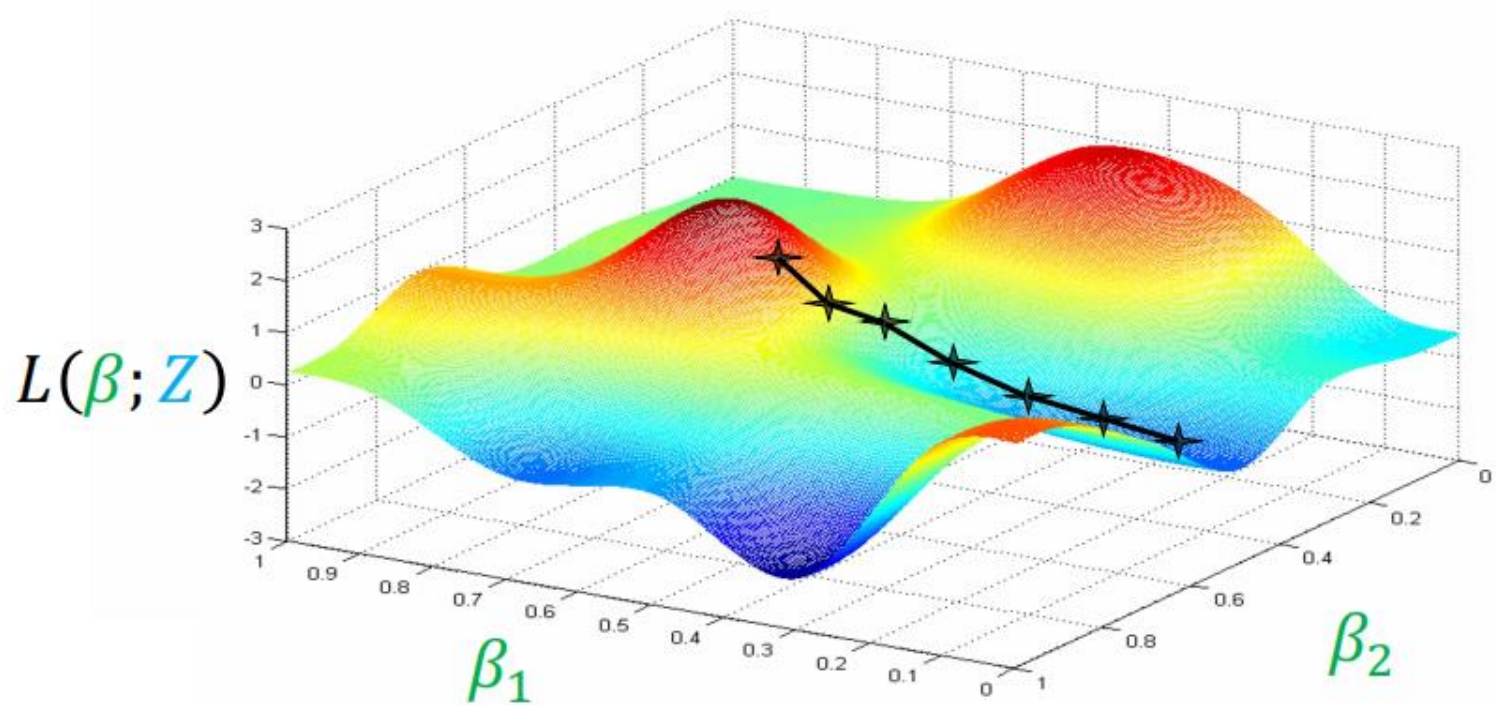
- **Gradient descent:** Update β based on **gradient** $\nabla_{\beta} L(\beta; Z)$ of $L(\beta; Z)$:

$$\beta_{t+1} \leftarrow \beta_t - \alpha \cdot \nabla_{\beta} L(\beta_t; Z)$$

- **Intuition:** The gradient is the direction along which $L(\beta; Z)$ changes most quickly as a function of β
- $\alpha \in \mathbb{R}$ is a hyperparameter called the **learning rate**
 - More on this later

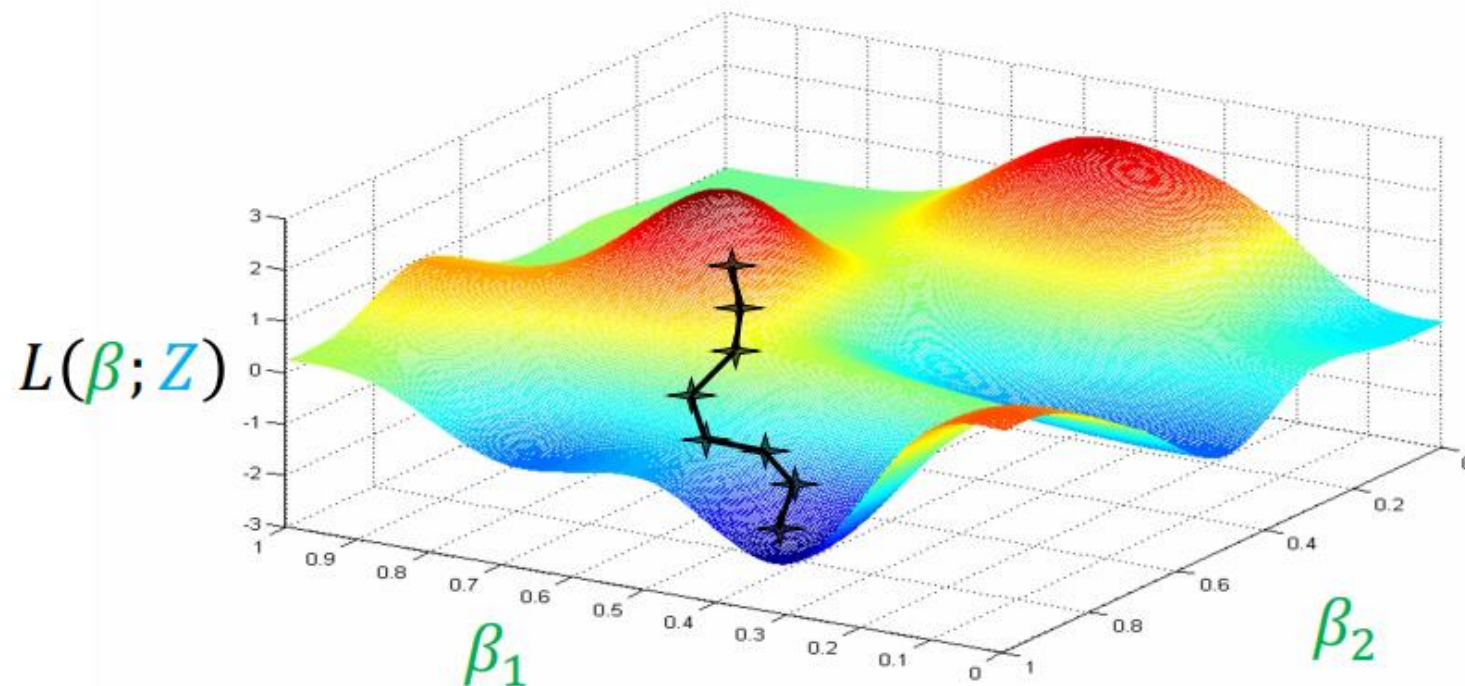
Strategy 2: Gradient Descent

- Choose initial value for β
- Until we reach a minimum:
 - Choose a new value for β to reduce $L(\beta; Z)$



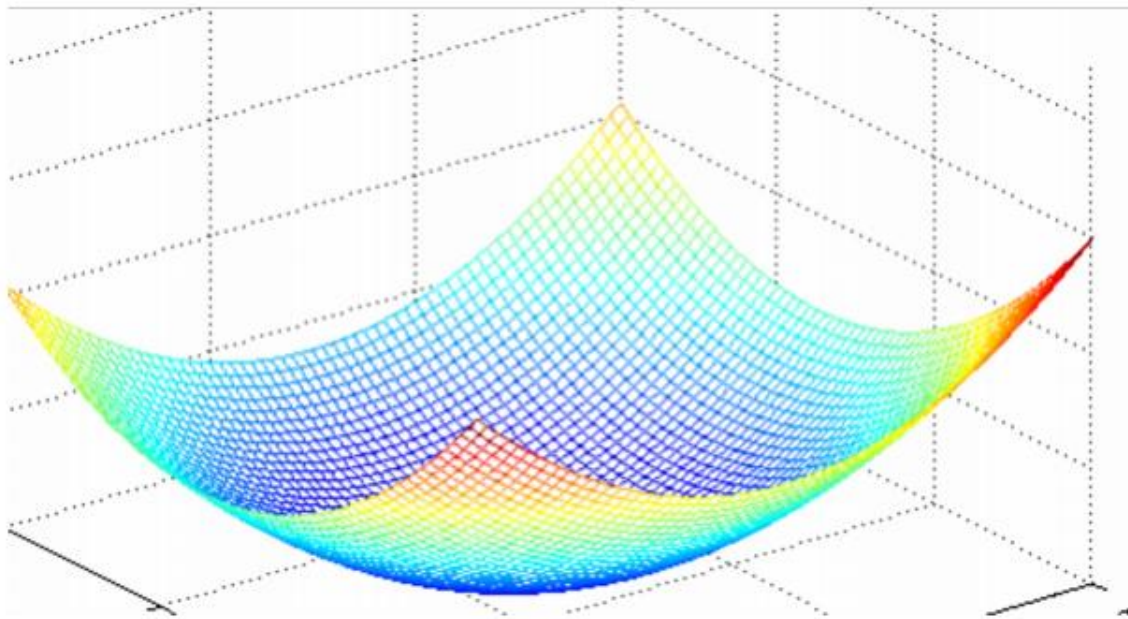
Strategy 2: Gradient Descent

- Choose initial value for β
- Until we reach a minimum:
 - Choose a new value for β to reduce $L(\beta; Z)$



Strategy 2: Gradient Descent

- Choose initial value for β
- Until we reach a minimum:
 - Choose a new value for β to reduce $L(\beta; Z)$



Linear regression loss is convex, so no local minima

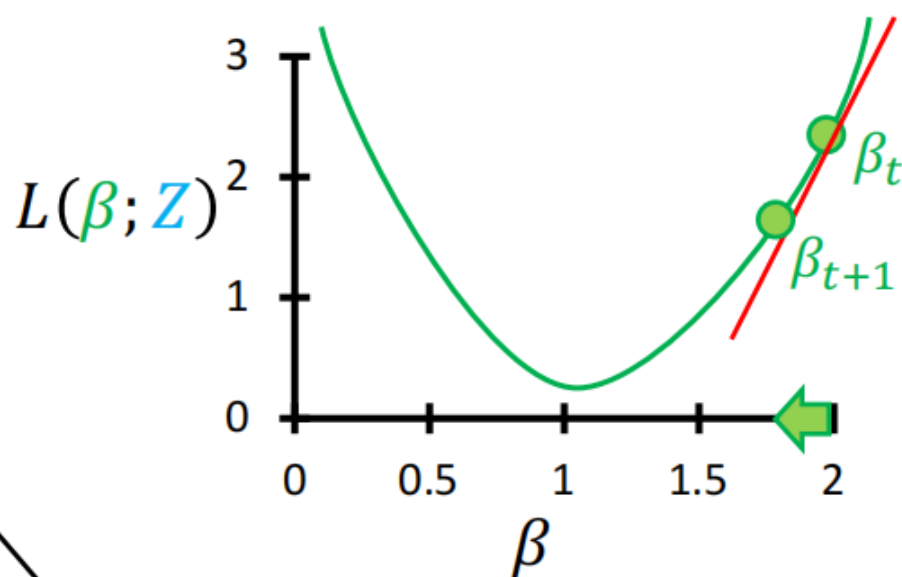
Strategy 2: Gradient Descent

Indexing iteration now, rather than parameter vector element

- Initialize $\beta_1 = \vec{0}$
- Repeat until convergence:

$$\beta_{t+1} \leftarrow \beta_t - \alpha \cdot \nabla_{\beta} L(\beta_t; \mathbf{Z})$$

- For linear regression, know the gradient from strategy 1



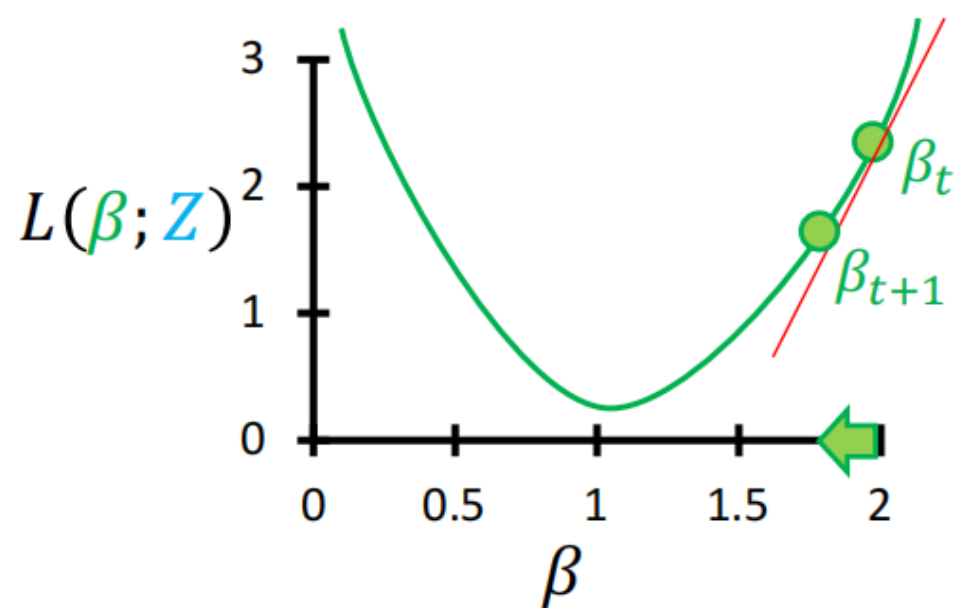
For in-place updates $\beta \leftarrow \beta - \alpha \cdot \nabla_{\beta} L(\beta; \mathbf{Z})$, compute all components of $\nabla_{\beta} L(\beta; \mathbf{Z})$ before modifying β

Strategy 2: Gradient Descent

- Initialize $\beta_1 = \vec{0}$
- Repeat until **convergence**:

$$\beta_{t+1} \leftarrow \beta_t - \alpha \cdot \nabla_{\beta} L(\beta_t; \mathbf{Z})$$

- For linear regression, know the gradient from strategy 1

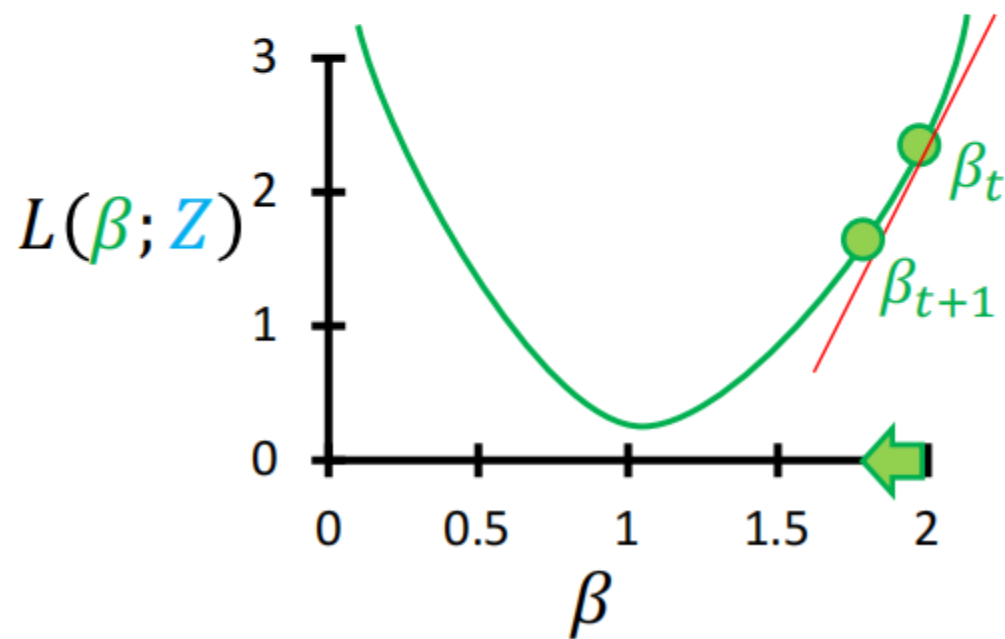


Strategy 2: Gradient Descent

- Initialize $\beta_1 = \vec{0}$
 - Repeat until $\|\beta_t - \beta_{t+1}\|_2 \leq \epsilon$:
- Hyperparameter defining convergence

$$\beta_{t+1} \leftarrow \beta_t - \alpha \cdot \nabla_{\beta} L(\beta_t; \mathbf{Z})$$

- For linear regression, know the gradient from strategy 1



Aside: Gradient As Sum of Sample-Wise Gradients

(Equivalent to our earlier matrix expression of gradient)

$$-\frac{2}{n}X^T Y + \frac{2}{n}X^T X \beta$$


- By linearity of the gradient, we have

$$\nabla_{\beta} L(\beta; Z) = \sum_{i=1}^n \nabla_{\beta} (y_i - \beta^T x_i)^2 = \sum_{i=1}^n 2(y_i - \beta^T x_i) x_i$$

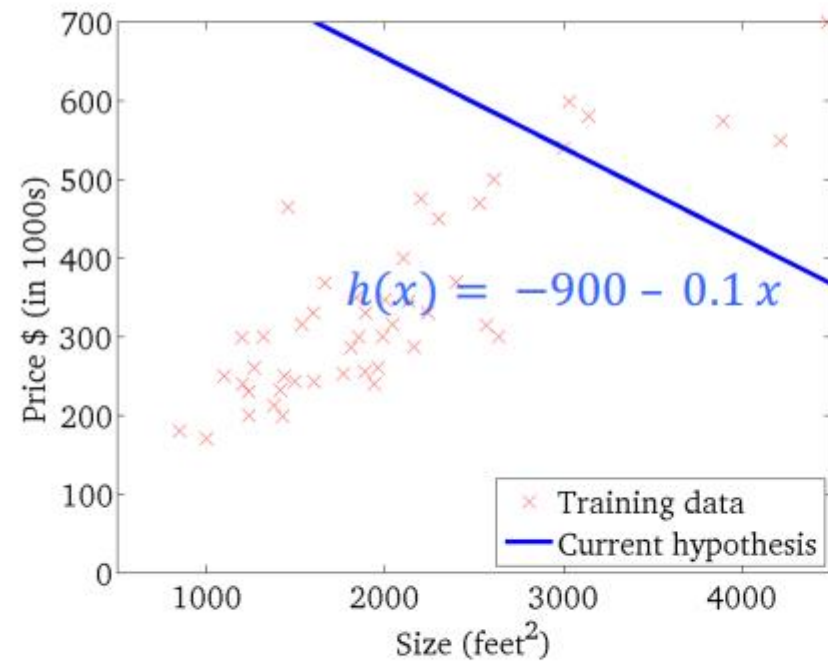
- The gradient term induced by a single training data sample is:

$$\nabla_{\beta} (y_i - \beta^T x_i)^2 = 2(y_i - \beta^T x_i) x_i$$

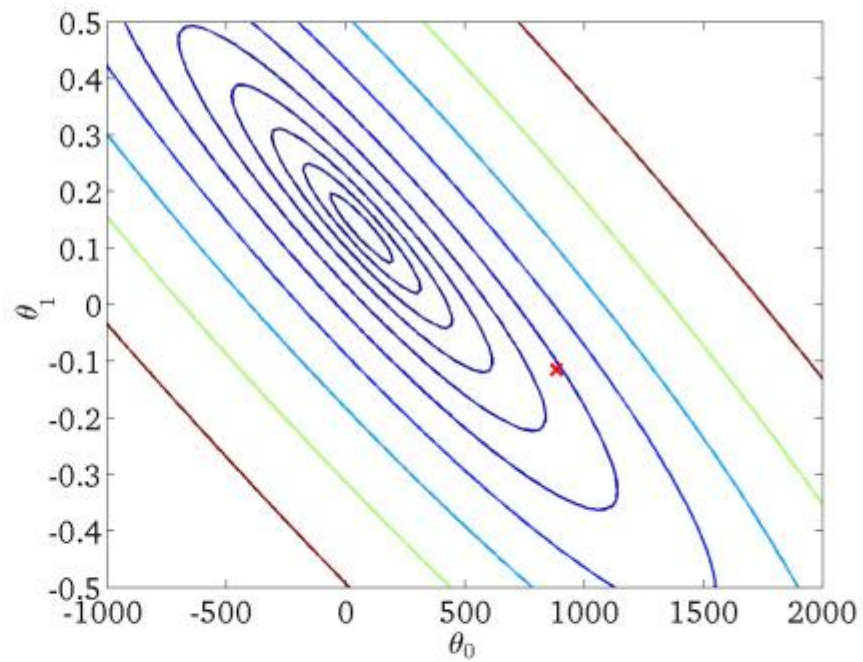
- I.e., the current error $y_i - \beta^T x_i$ times the feature vector x_i

“Large error samples induce large changes to β , proportional to their feature values.”

Strategy 2: Gradient Descent

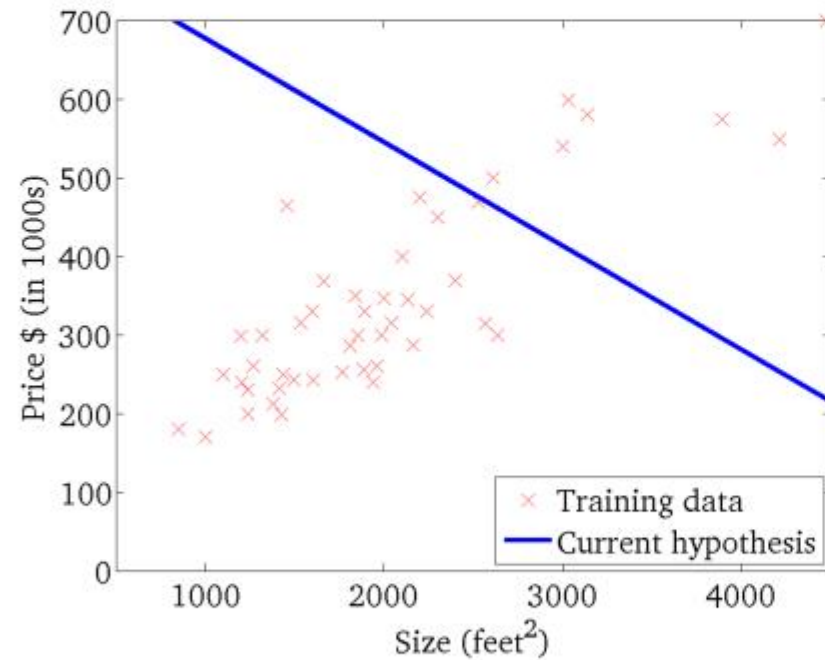


$$f_{\beta}(x)$$

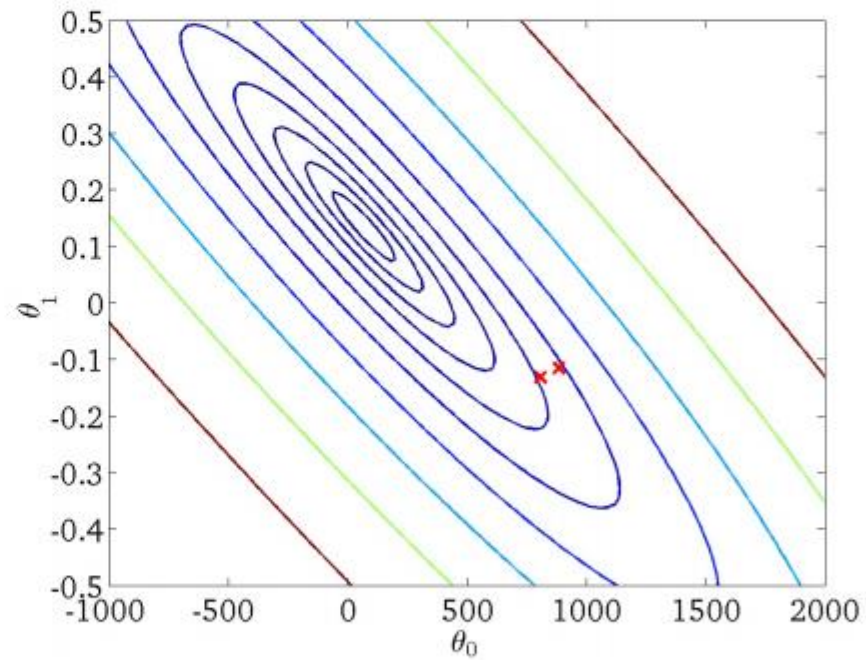


$$L(\beta; Z)$$

Strategy 2: Gradient Descent

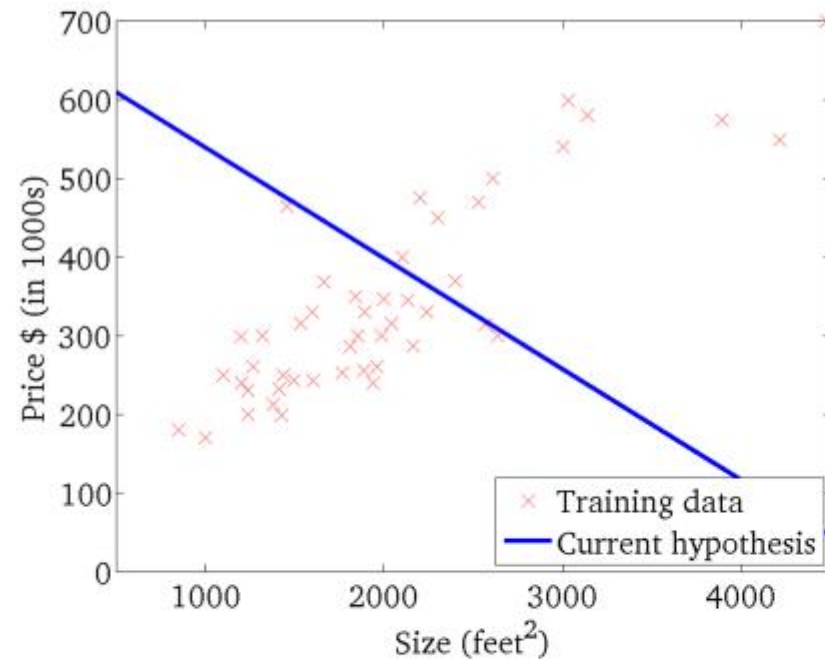


$$f_{\beta}(x)$$

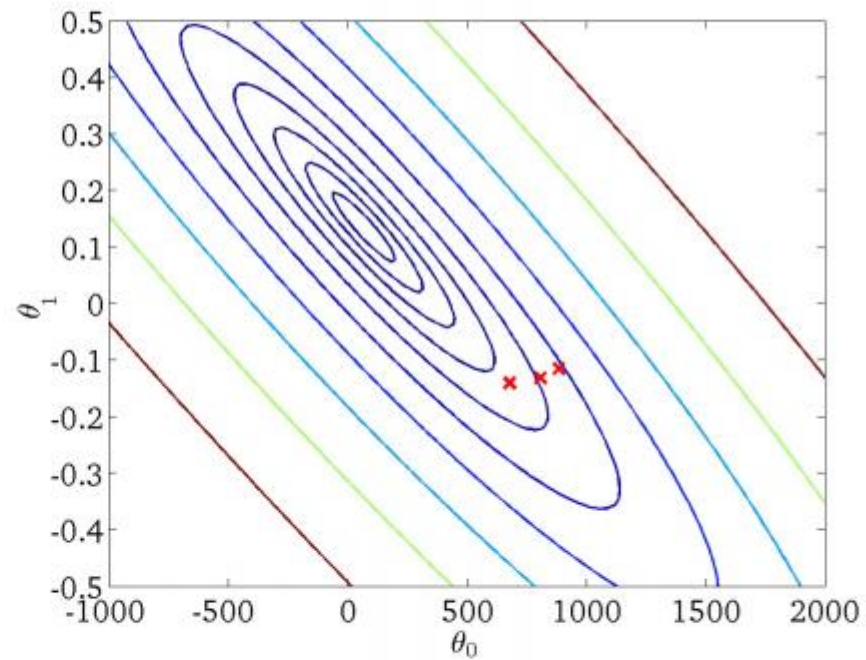


$$L(\beta; Z)$$

Strategy 2: Gradient Descent

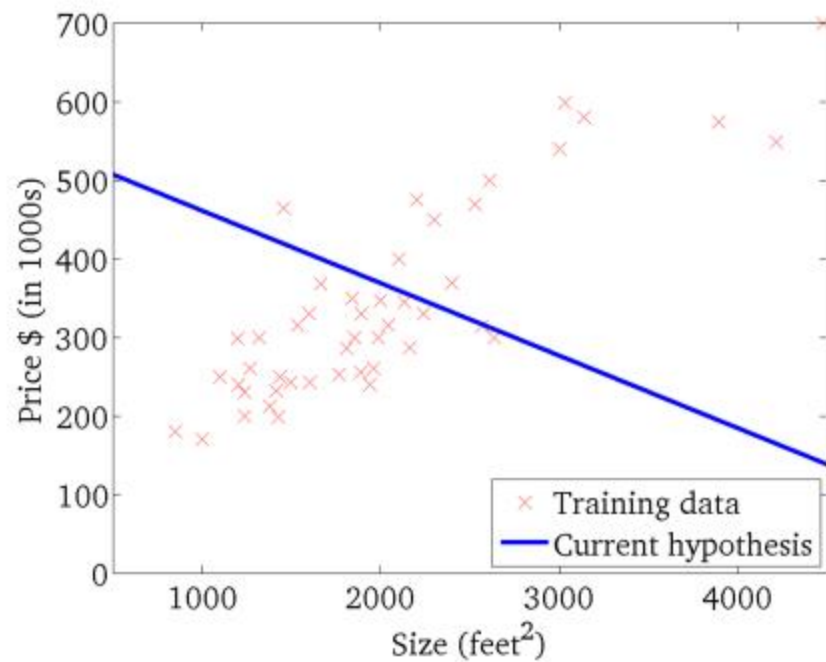


$$f_{\beta}(x)$$

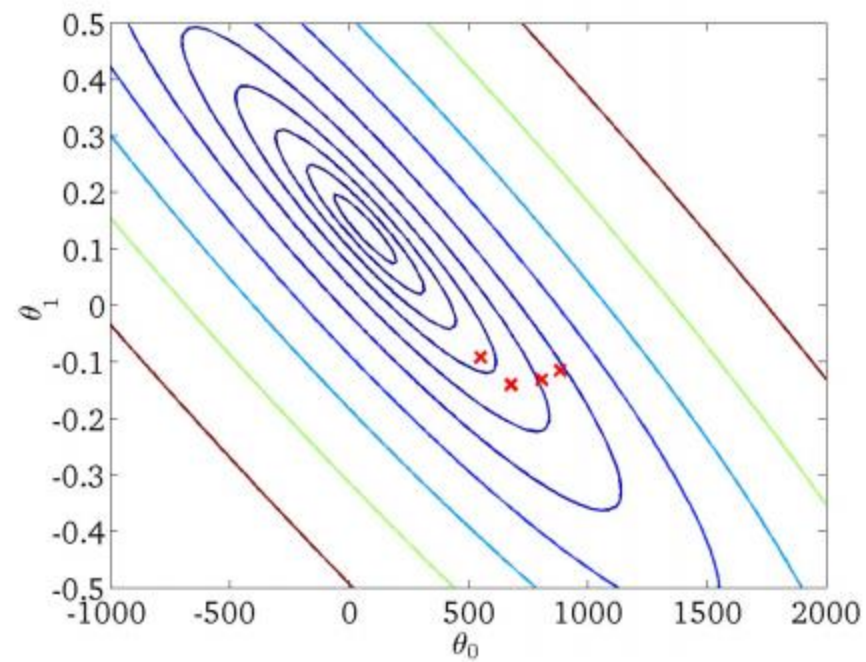


$$L(\beta; Z)$$

Strategy 2: Gradient Descent

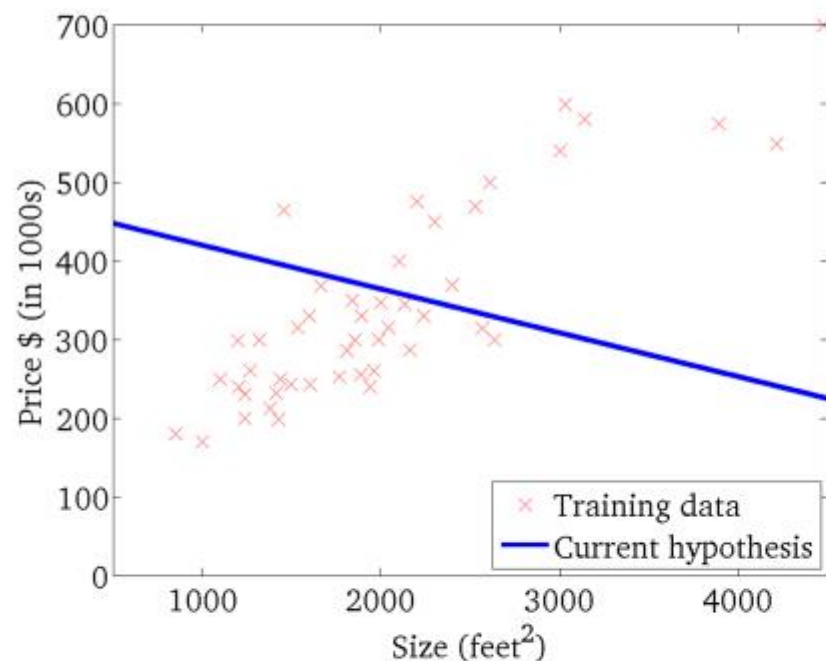


$$f_{\beta}(x)$$

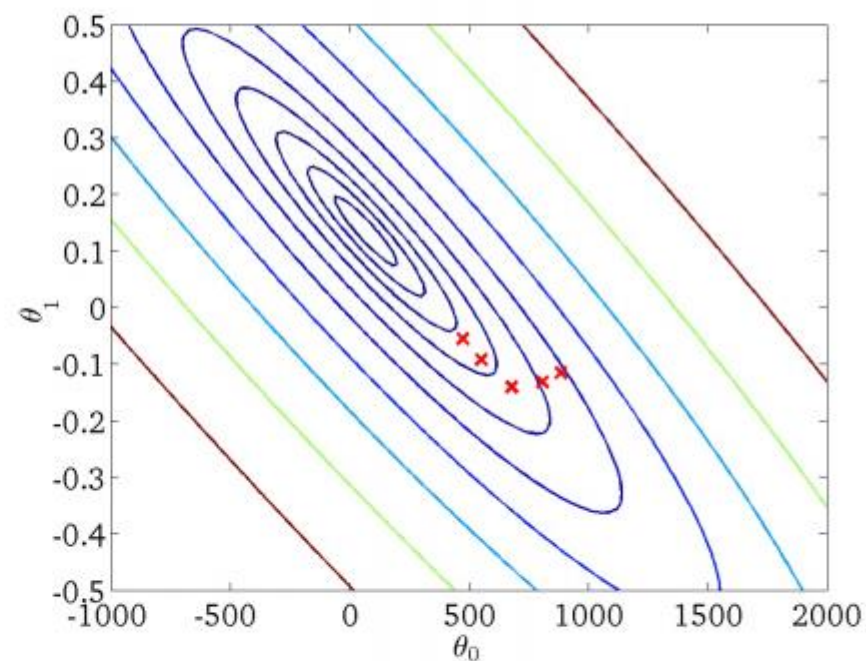


$$L(\beta; Z)$$

Strategy 2: Gradient Descent

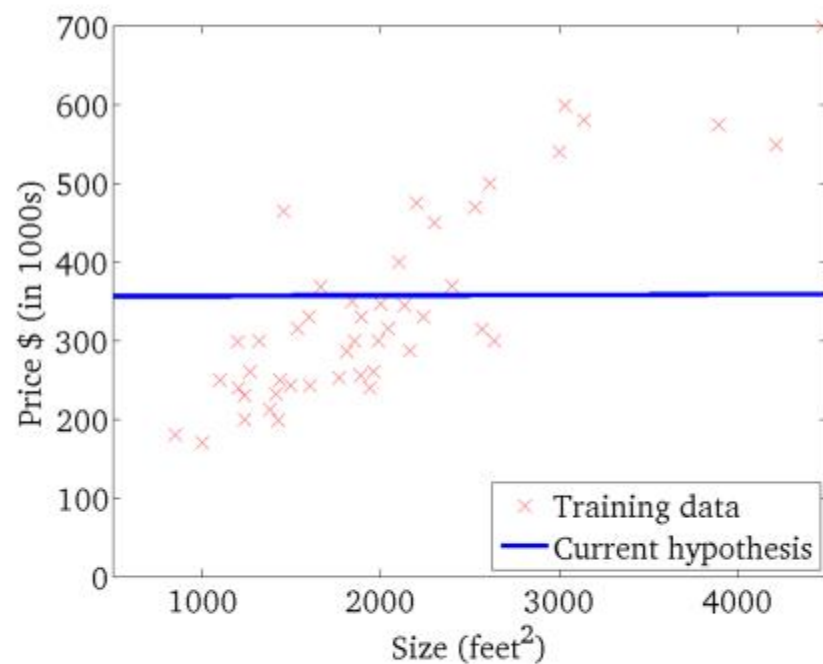


$$f_{\beta}(x)$$

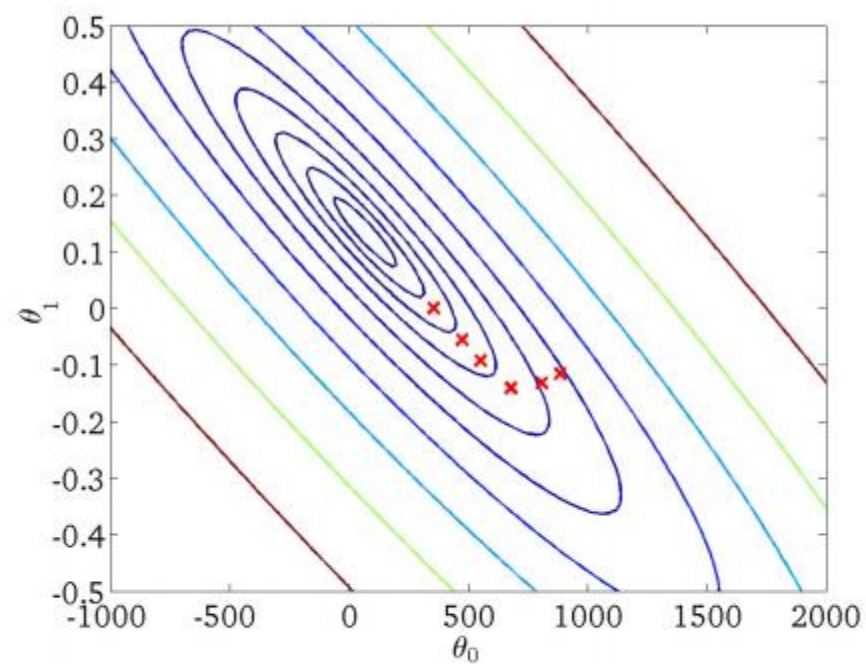


$$L(\beta; Z)$$

Strategy 2: Gradient Descent

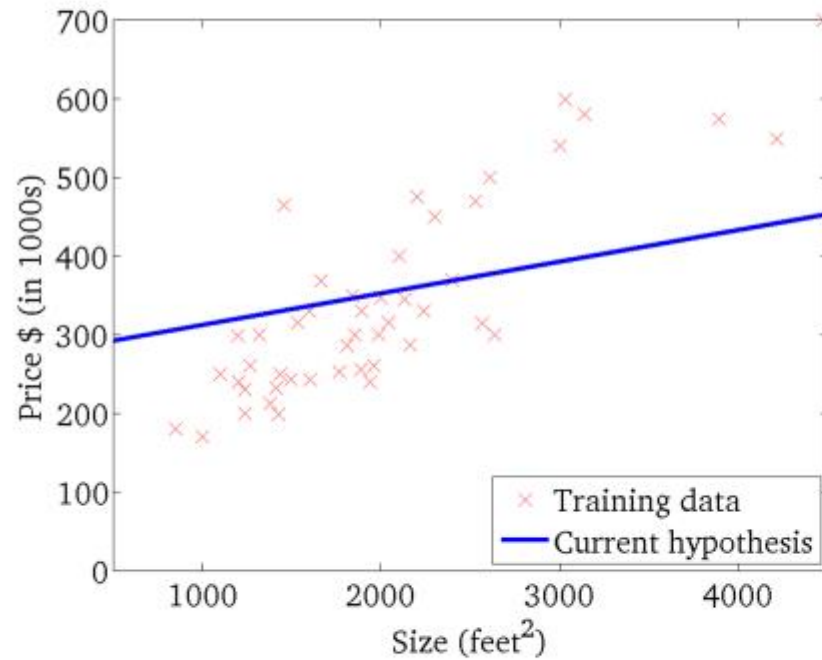


$$f_{\beta}(x)$$

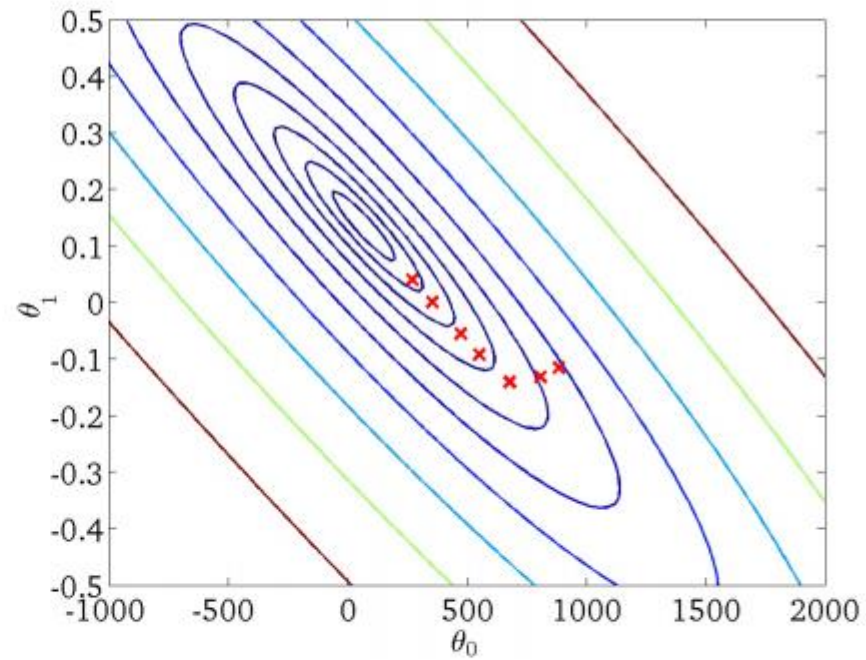


$$L(\beta; Z)$$

Strategy 2: Gradient Descent

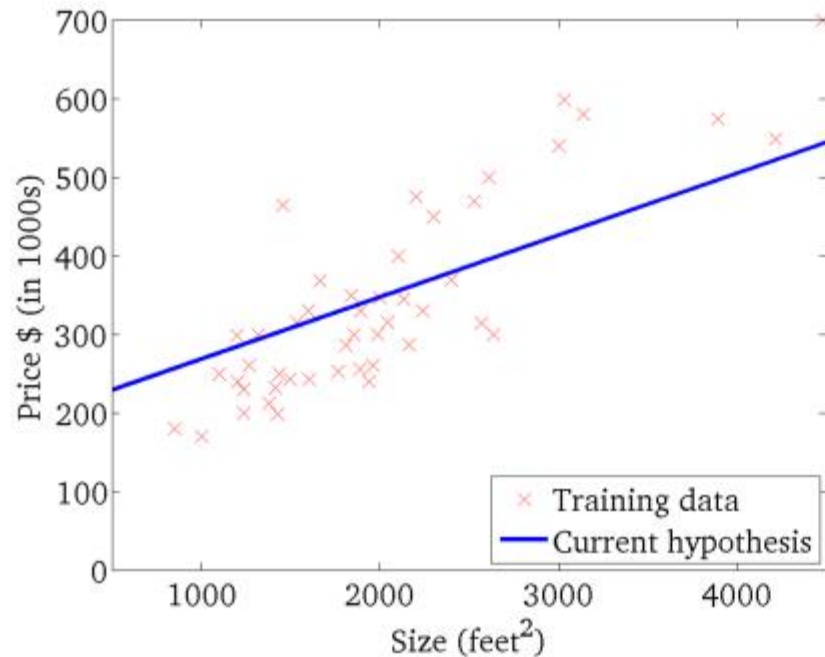


$$f_{\beta}(x)$$

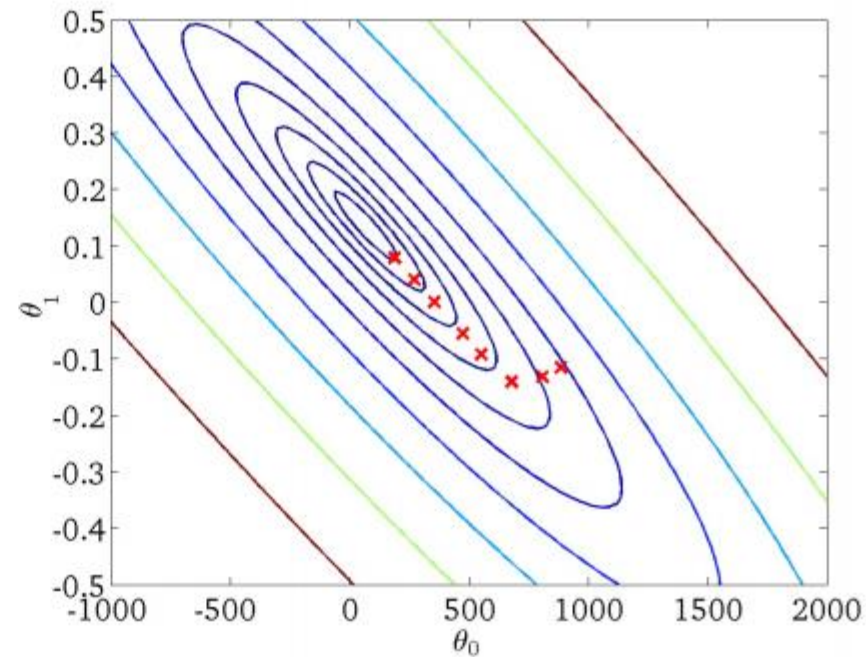


$$L(\beta; Z)$$

Strategy 2: Gradient Descent

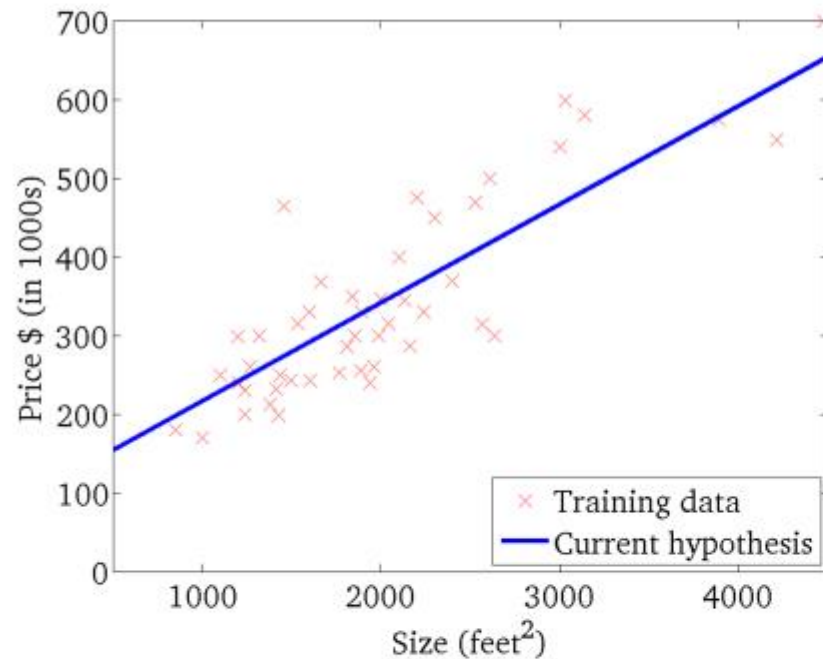


$$f_{\beta}(x)$$



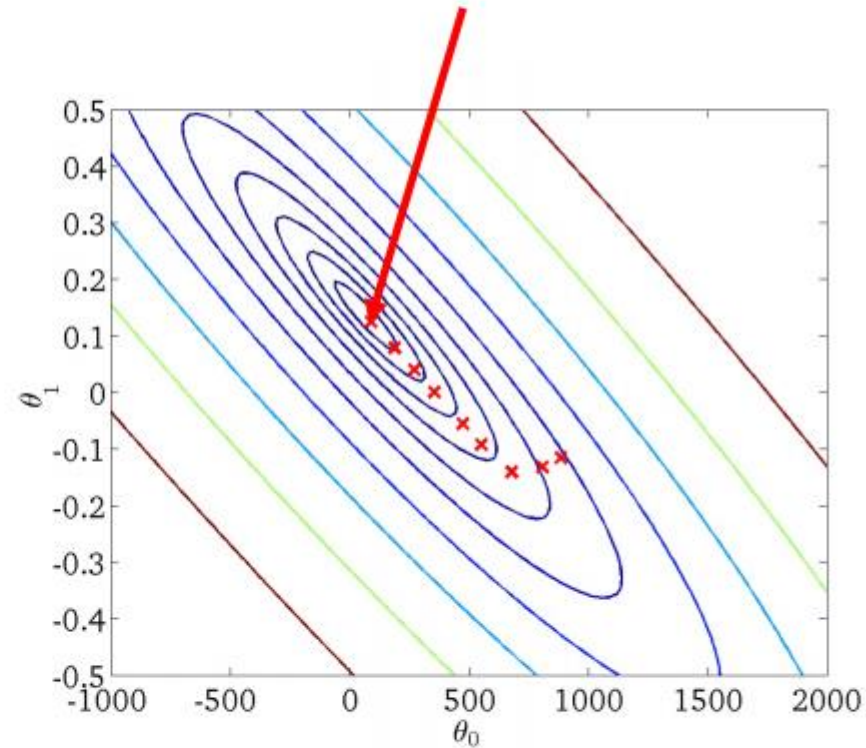
$$L(\beta; Z)$$

Strategy 2: Gradient Descent



$$f_{\beta}(x)$$

Minimizer of loss function



$$L(\beta; Z)$$

插播~~

Loss Minimization View of ML

- **Three** design decisions

- **Model family:** What are the candidate models f ? (E.g., linear functions)
- **Loss function:** How to define “approximating”? (E.g., MSE loss)
- **Optimizer:** How do we minimize the loss? (E.g., gradient descent)